

RGB to CMYK with TIC Control

Print-Safe Color with Minimal Visual Loss

Goal : Convert display RGB images into a CMYK representation, visualize channel-wise ink usage, limit total ink coverage, reconstruct RGB, and quantify the quality loss.

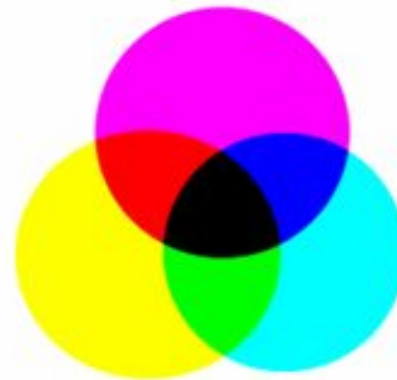
RGB and CMYK Definitions

RGB: RGB is a color model that consists of 3 major colors that is Red, Blue and Green. Using these three primaries it is possible to create a color reproduction system that can be used to generate a wide variety of colours.

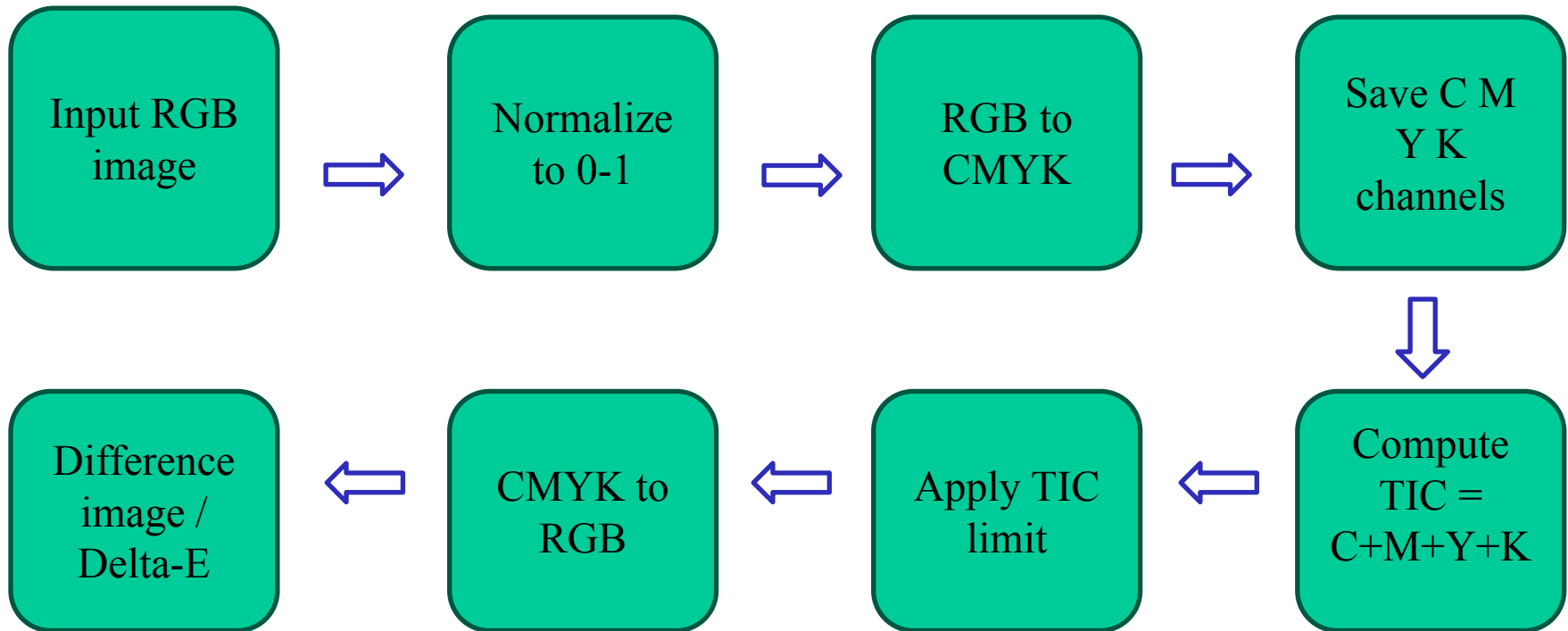
Modern Examples are: LED Screens, plasma displays and cinematography.



CMYK: It consists of 3 major colors that is Cyan, Magenta and Yellow and a key (Black). This is a subtractive color gamut. This color model can produce a wide spectrum for printing technologies.



Workflow Block Diagram



Numerical Understanding

RGB to CMYK Conversion

- First, RGB values are usually normalized from 0–255 into 0–1. Then the black component is estimated as **$K=1-\max(R,G,B)$** and the cyan, magenta, and yellow values are computed from the remaining color after removing black. A common formula is:
- $C=(1-R-K)/(1-K)$
- $M=(1-G-K)/(1-K)$
- $Y=(1-B-K)/(1-K)$
- **Total Ink Coverage(TIC)** = $C+M+Y+K$

CMYK to RGB Conversion

- To go back, the channels are combined into visible light values again. A common approximation is:
- $R=255 \times (1-C) \times (1-K)$
- $G=255 \times (1-M) \times (1-K)$
- $B=255 \times (1-Y) \times (1-K)$
- This is the intuitive inverse of CMYK printing: less ink means more light is reflected, so the RGB values become brighter.

Numerical Understanding

Delta CIEDE2000 Formula

- **Sl** – Defines how bright or Dark
- **Sc** - Adjusts the chroma difference(Vivid or dull)
- **Sh** - Adjust Hue difference (Which color family it belongs to like R,G,B)
- **Delta E** - There is a huge difference in the original and reconstructed image.
- **Delta e** – Reconstructed image is closer to the original image
- **Rt** - a correction term for certain hue regions, especially around blue.

$$\Delta E_{00} = \sqrt{\left(\frac{\Delta L^*}{k_L \cdot S_L}\right)^2 + \left(\frac{\Delta C^*}{k_C \cdot S_C}\right)^2 + \left(\frac{\Delta H^*}{k_H \cdot S_H}\right)^2 + \left(R_T \left(\frac{\Delta C^*}{k_C \cdot S_C}\right) \left(\frac{\Delta H^*}{k_H \cdot S_H}\right)\right)}$$

where

$$L^* = L^* \quad a^* = a^*(1+G) \quad G = 0.5 \left(1 - \sqrt{\frac{\bar{C}_{ab}^{*7}}{\bar{C}_{ab}^{*7} + 25^7}} \right)$$

$$b^* = b^* \quad C^* = \sqrt{(a^*)^2 + (b^*)^2} \quad h^* = \tan^{-1} \left(\frac{b^*}{a^*} \right)$$

The weighting coefficients S_L , S_C , and S_H are defined by the following equations

$$S_L = 1 + \frac{0.015 (\bar{L}^* - 50)^2}{\sqrt{20 + (\bar{L}^* - 50)^2}} \quad S_C = 1 + 0.045 \bar{C}^* \quad S_H = 1 + 0.015 \bar{C}^* T$$

$$T = 1 - 0.17 \cos(\bar{h}^* - 30) + 0.24 \cos(2\bar{h}^*) + 0.32 \cos(3\bar{h}^* + 6) - 0.20 \cos(4\bar{h}^* - 63)$$

The rotation factor R_T is determined as shown below. (The hue angle and θ are in degrees.)

$$R_T = -\sin(2\Delta\theta) R_C \quad \Delta\theta = 30 \exp \left(- \left(\frac{\bar{h}^* - 275}{25} \right)^2 \right)$$

$$R_C = 2 \sqrt{\frac{\bar{C}^{*7}}{\bar{C}^{*7} + 25^7}}$$

RGB as a Input

Original RGB Image



CMYK Channels

CMYK Channels (Grayscale)

Cyan



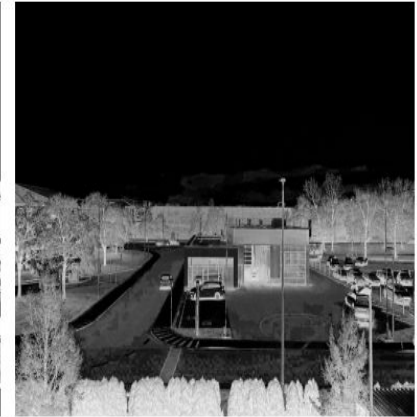
Magenta



Yellow



Black (Key)

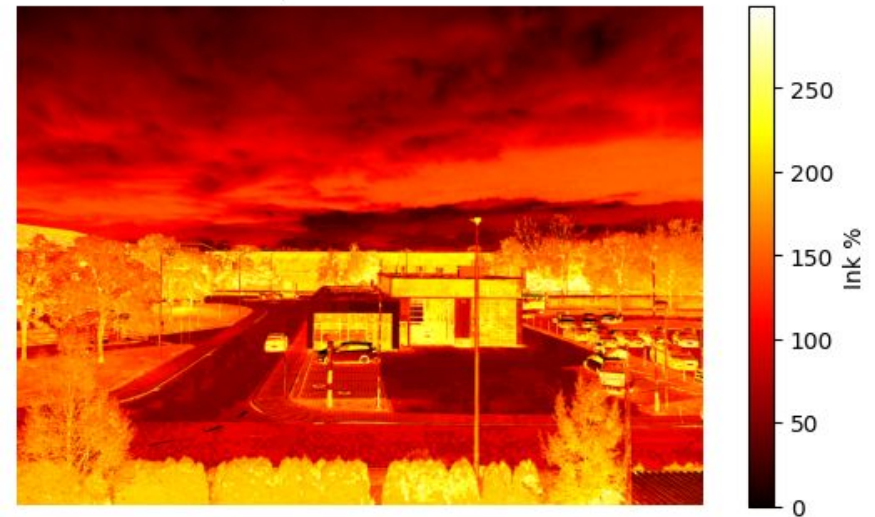


Ink Consumption

Ink Consumption after 250% Limit



Ink Consumption after 300% Limit



Heatmap analysis after the limit

Logic

```
#saving cmyk channels as grayscales
for ch, name in zip(channels, channel_names):
    ch_uint8 = (ch * 255).astype(np.uint8)
    ch_img = Image.fromarray(ch_uint8, mode='L')
    filename = os.path.join(save_folder, f'channel_{name.split()[0]}.png')
    ch_img.save(filename)
    print(f"Saved: {filename}")
```

```
/tmp/ipykernel_2187/860260907.py:4: DeprecationWarning: 'mode' parameter is deprecated and will be removed in Pillow 13 (2026-10-15)
    ch_img = Image.fromarray(ch_uint8, mode='L')
Saved: /content/channel_Cyan.png
Saved: /content/channel_Magenta.png
Saved: /content/channel_Yellow.png
Saved: /content/channel_Black.png
```

```
#apply and visualize both limits
for limit in [250, 300]:
    C_l, M_l, Y_l, K_l = apply_ink_limit(C, M, Y, K, limit)
    ink_limited = (C_l + M_l + Y_l + K_l) * 100

    plt.figure(figsize=(7, 4))
    plt.imshow(ink_limited, cmap='hot')
    plt.colorbar(label='Ink %')
    plt.title(f"Ink Consumption after {limit}% Limit")
    plt.axis('off')
    plt.show()

    print(f"[{limit}% limit] Max: {ink_limited.max():.1f}% | Avg: {ink_limited.mean():.1f}%")
```

Reconstructed image as a Output

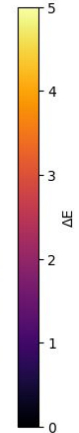
Reconstructed (250%)



Pixel Difference (scaled) (250%)



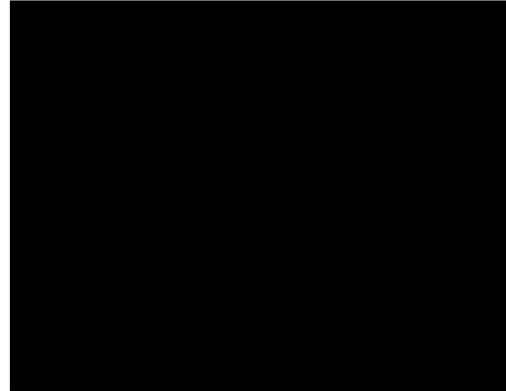
Delta-E 2000 Map (250%)



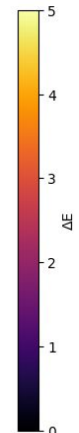
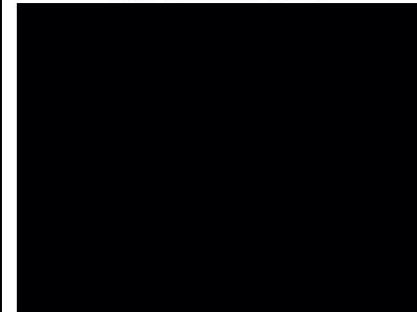
Reconstructed (300%)



Pixel Difference (scaled) (300%)



Delta-E 2000 Map (300%)



[250%]	Avg ΔE : 0.0054	Max ΔE : 2.12
[300%]	Avg ΔE : 0.0000	Max ΔE : 0.00

Logic

```
# to visualize raw and reconstructed image after processing the whole pipeline
fig, axes = plt.subplots(2, 3, figsize=(16, 9))

for row, (recon, dmap, label) in enumerate([
    (rgb_reconstructed_250, delta_map_250, "250%"),
    (rgb_reconstructed_300, delta_map_300, "300%"),
]):

    diff = np.abs(img_rgb - recon)

    # Use FIXED scaling (not per-image normalization)
    diff_scaled = np.clip(diff * 50, 0, 1) # adjust 50 → 20-100 if needed

    # reconstructing the image
    axes[row, 0].imshow(np.clip(recon, 0, 1))
    axes[row, 0].set_title(f"Reconstructed ({label})")
    axes[row, 0].axis('off')

    # this will show the pixel difference
    axes[row, 1].imshow(diff_scaled)
    axes[row, 1].set_title(f"Pixel Difference (scaled) ({label})")
    axes[row, 1].axis('off')

    # delta e map
    im = axes[row, 2].imshow(dmap, cmap='inferno', vmin=0, vmax=5)
    axes[row, 2].set_title(f"Delta-E 2000 Map ({label})")
    axes[row, 2].axis('off')

plt.colorbar(im, ax=axes[row, 2], label='ΔE')
```

Verdict

- **Issue** - RGB images are made for screens, but printers use CMYK. A direct conversion may create too much total ink in some areas.
- **Result** - The output becomes more suitable for printing because the total ink is controlled.
- **Takeaway** - Reduced ink safely without introducing too much visible color error.

Citations and References

- RGB image:
<https://kharshit.github.io/blog/2020/01/17/color-and-color-spaces-in-computer-vision>
- RGB and CMYK: Westland S & Cheung V, 2012. RGB Systems, cmyk Systems in Handbook of Visual Display Technology
- CIA2000 Formula:
<https://www.konicaminolta.com/instruments/knowledge/color/part4/09.html>
- <https://arrowpix.in/rgb-to-cmyk-converter>
- Acquisition of Color Reproduction Technique based on Deep Learning Using a Database of Color-converted Images in the Printing Industry Ikumi Hirose† and Ryosuke Yabe†
- <https://colourlab.readthedocs.io/en/latest/> - Website
- <https://github.com/ifarup/colourlab> - Github
- <https://techkon.datacolor.com/demystifying-the-cie-delta-e-2000-formula/#:~:text=The%20OCIE%20dE2000%20formula%20takes,along%20with%20the%20weighting%20functions>
- https://python-colormath.readthedocs.io/_/downloads/en/2.2.0/pdf/